Animator Controller & Animation Structure – Summary

This document summarizes the **final, production-ready standard** for Animator Controllers, Overrides, and Animation organization for **Hero** and **Enemy** characters.

Core Principles

1. **Animator Controller = Behavior + Weapon Archetype**
2. **Animator Override Controller = Visual Differences** (race, gender, model, proportions)
3. **FSM & Turn systems are animation-agnostic**
4. **Weapon type defines attack logic and timing** – not race or gender

---

High-Level Responsibility Split

Layer	Responsibility
Turn FSM	Time ownership, turn order
Entity FSM (Hero / Enemy)	Behavior flow (Idle → Ready → Attack → Idle)
Animator Controller	Attack structure, combos, hit timing
Animator Override	Visual differences only
Animation Clips	Raw motion data

Folder Structure (Root)

```
Assets/
 └ Gameplay/
    └ Animation/
       ├ Controllers/
       ├ Overrides/
       ├ Clips/
       └ Avatars/
```

Animator Controllers

Enemy Controllers

```
Controllers/Enemy/
 ├─ Melee/
 │   ├─ Enemy_Melee_Sword.controller
 │   ├─ Enemy_Melee_Axe.controller
 │   └─ Enemy_Melee_Spear.controller
 ├─ Ranged/
 │   ├─ Enemy_Ranged_Bow.controller
 │   └─ Enemy_Ranged_Crossbow.controller
 └─ Caster/
     ├─ Enemy_Caster_Staff.controller
     └─ Enemy_Caster_Book.controller
```

Naming rule:

```
Enemy_<CombatType>_<WeaponArchetype>.controller
```

Hero Controllers

```
Controllers/Hero/
 ├─ Warrior/
 │   ├─ Hero_Warrior_Sword.controller
 │   ├─ Hero_Warrior_Axe.controller
 │   └─ Hero_Warrior_Spear.controller
 ├─ Wizard/
 │   ├─ Hero_Wizard_Staff.controller
 │   └─ Hero_Wizard_Book.controller
 └─ Archer/
     ├─ Hero_Archer_Bow.controller
     └─ Hero_Archer_Crossbow.controller
```

Naming rule:

```
Hero_<Class>_<WeaponArchetype>.controller
```

Animator Override Controllers

Overrides handle **race, gender, model, proportions**.

```
Overrides/
 ├─ Hero/
 │   └─ Warrior/
 │       └─ Sword/
 │           ├─ Hero_Warrior_Sword_Human_Male.override
 │           ├─ Hero_Warrior_Sword_Elf_Female.override
 │           └─ Hero_Warrior_Sword_Orc_Male.override
 └─ Enemy/
     └─ Melee/
         └─ Axe/
             ├─ Enemy_Melee_Axe_Orc_Male.override
             └─ Enemy_Melee_Axe_Dwarf_Male.override
```

Naming rule:

```
<ControllerName>_<Race>_<Gender>.override
```

Animation Clips

```
Clips/
 ├─ Shared/
 │   ├─ Idle.anim
 │   ├─ Run.anim
 │   ├─ HitReact.anim
 │   └─ Die.anim
 ├─ Hero/
 │   ├─ Warrior/
 │   │   ├─ Attack_01.anim
 │   │   ├─ Attack_02.anim
 │   │   └─ Skill_Ultimate.anim
 │   └─ Wizard/
 └─ Enemy/
     ├─ Melee/
     ├─ Ranged/
     └─ Boss/
```

Important: - Clip names must be **semantic** - Do **NOT** include race or gender in clip names

Avatar Assets (Humanoid Rigs)

```
Avatars/
 ├─ Human.avatar
 ├─ Elf.avatar
 ├─ Orc.avatar
 └─ Skeleton.avatar
```

FSM & Code Integration

FSM always uses semantic parameters only:

```
animator.SetTrigger("DoAttack");
animator.SetInteger("AttackVariant", variant);
```

- FSM does **not** know race, gender, or weapon
- Weapon-specific logic lives in the Animator Controller

Hero vs Enemy – Key Differences

Topic	Hero	Enemy
Controller granularity	Class + Weapon	Weapon + Behavior
Combo depth	High	Low–Medium
Visual priority	Very High	Medium
Override usage	Extensive	Moderate

Final Takeaway

- **Weapon archetype defines animator behavior**
- **Class modifies behavior only for heroes**
- **Overrides handle all visual diversity**
- **FSM and Turn systems remain clean and reusable**

This structure scales safely to dozens of heroes and hundreds of enemies without refactor pressure.